

## TRANS - idea

# E-facture — service de vérification et de conversion des factures électroniques

### Le problème

À partir du 1er septembre 2026, toutes les entreprises françaises assujetties à la TVA devront être en mesure de **recevoir** des factures électroniques ; les grandes entreprises et les ETI devront également en **émettre**. Le format s'appelle Factur-X : un PDF classique dans lequel est intégré un XML contenant les données structurées.

Des millions de petites entreprises n'y sont pas préparées. Leurs logiciels de facturation produisent des PDF classiques, et les fichiers XML qu'elles reçoivent leur sont tout simplement illisibles. Des outils de vérification existent, mais ils renvoient des erreurs du type [BR-CO-10] Sum of Invoice line net amount MUST be equal... avec une adresse XPath — incompréhensible pour quelqu'un qui découvre le sujet.

### Ce que nous construisons

Un service web multi-locataire destiné aux cabinets comptables et à leurs clients.

Le cabinet crée une organisation et y invite ses clients. Le client dépose ses factures, le comptable les voit arriver en temps réel, la vérification passe par une file d'attente, et les statuts sont diffusés en WebSocket à tous ceux qui consultent le cabinet.

Trois fonctions principales :

1. **Vérification** — on dépose une facture, on obtient un rapport clair en français : « Le total HT indiqué est de 1 250,00 €, mais la somme des lignes donne 1 245,00 €, écart de 5,00 € », au lieu d'un code de règle.
2. **Lecture** — un XML brut arrive, nous l'affichons sous la forme d'une facture normalement lisible.
3. **Conversion** — un PDF classique devient un Factur-X valide. Les champs sont extraits automatiquement, l'utilisateur les vérifie et les corrige dans un formulaire avant de valider.

### Architecture

**Java (Spring Boot)** — le moteur de conformité en quatre couches de contrôle : PDF/A-3, XSD, Schematron EN 16931, profils Factur-X. Ainsi que les organisations, les rôles et les permissions.

**Python (FastAPI)** — l'extraction des champs depuis le PDF et la génération du

Factur-X.

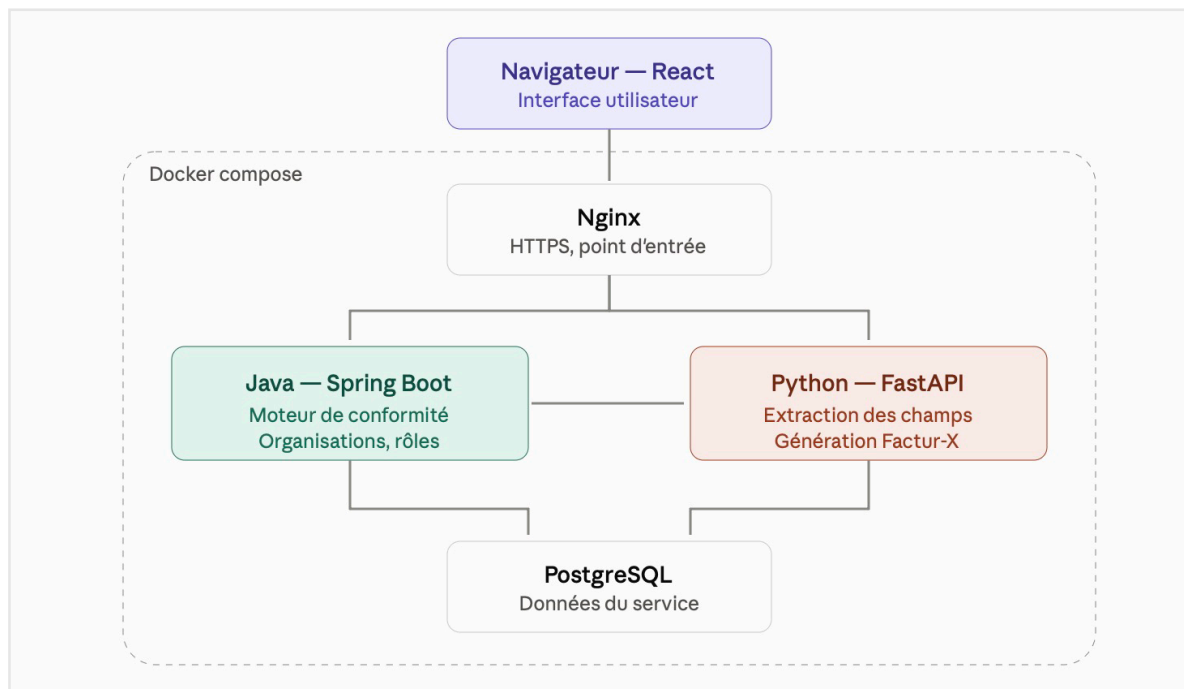
**React** — l'interface.

**PostgreSQL** — organisations, membres, documents, rapports, catalogue de règles, journal d'audit.

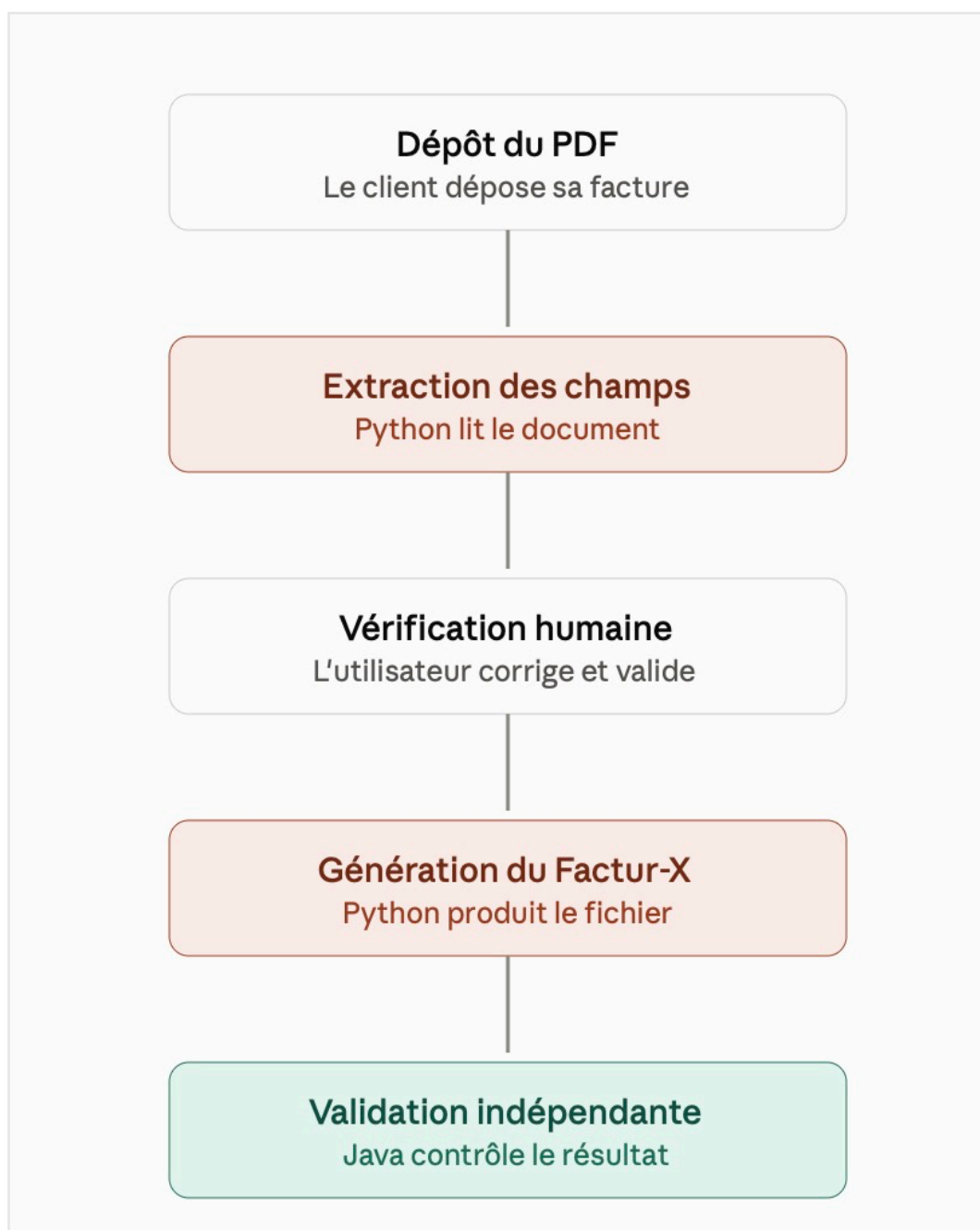
Le point élégant : c'est Python qui génère et Java qui valide, de façon indépendante. En CI, le build échoue si une seule facture générée s'avère non conforme.

Le choix de Java n'est pas arbitraire : tout l'écosystème de la norme EN 16931 (Mustang, Saxon, veraPDF) vit sur la JVM.

## STACK:



How it works:



### **nos Modules — 18 points pour 14 requis**

**Web — 7** : frameworks front et back (2), API publique avec clé et rate limiting (2), ORM (1), système de gestion de fichiers (1), recherche avancée (1).

**User Management — 5** : système d'organisations (2), système de permissions avancé (2), 2FA (1).

**DevOps — 2** : backend en microservices (2) — nous construisons de toute façon deux services indépendants.

**Data & Analytics — 2** : export et import de données (1), conformité RGPD (1).

**Modules of choice — 2** : le moteur de conformité Factur-X en tant que module Major personnalisé. C'est là que compte tout le travail métier — les quatre couches de validation, le catalogue de règles, la traduction du diagnostic en langage compréhensible.

Une marge de 4 points, au cas où un module ne serait pas validé lors de l'évaluation.

## Partie obligatoire du sujet

Application web avec frontend, backend et base de données. Inscription et connexion par email avec mots de passe hachés. Validation des formulaires côté client et côté serveur. HTTPS. Docker, lancement en une seule commande. Pages Politique de confidentialité et Conditions d'utilisation accessibles depuis le footer — leur absence entraîne le rejet du projet. Support de plusieurs utilisateurs simultanés, sans race conditions ni corruption de données.

## Ce que nous ne faisons pas

Aucune intégration avec Chorus Pro ni avec les plateformes agréées — cela demande une certification. Uniquement le format CII, pas UBL. Génération limitée à deux profils sur cinq. Ni chat ni système d'amis — dans un outil B2B destiné aux comptables, cela paraîtrait artificiel, et on nous le reprochera à l'évaluation.

## Rôles dans l'équipe

Product Owner

## Project Manager

Technical Lead et développeurs, comme l'exige le sujet.

Tous les rôles sont documentés dans le README, et chacun doit pouvoir expliquer n'importe quelle partie du projet.

Répartition proposée : le moteur de validation Java et le catalogue de règles ;

l'extraction et la génération en Python ; le frontend et le temps réel ;

l'infrastructure, les permissions et les organisations. Dès les premiers jours, nous figeons le contrat OpenAPI entre les services — ensuite chacun avance de son côté.

## À propos de l'usage de l'IA

Le sujet exige une description honnête dans le README de l'usage qui a été fait de l'IA.

Et lors de l'évaluation, on nous demandera d'expliquer n'importe quelle portion de code, ainsi que d'apporter une petite modification sur place.

Le moteur de validation et le catalogue de règles, nous les écrivons nous-mêmes — c'est ce que nous aurons à défendre.

## Pourquoi ce projet

Un vrai domaine métier avec une vraie échéance, au lieu d'un exercice académique. Sur un CV, il se présente avec des chiffres : précision de l'extraction, latence, couverture des règles. Et il y a une réelle chance d'obtenir de premiers utilisateurs — le segment est immense et totalement non préparé.

////////////////////////////////////  
////////////////////////////////////